SIET CSE DEPARTMENT PLATFORM

MEENA — IMPLEMENTATION GUIDE

HOD Portal • Authentication • RLS • API Guards • Approvals • Records • Reviews • Reports

Goal: turn the existing frontend prototype into a secure, database-backed HOD workflow.

Based on PRD v2.0 and API Contract v2.0.

1. How to use this guide

Work in small steps. Do not build everything at once. For each step: read the task → use the prompt → inspect the result → test it → only then continue.

Golden rule: main must always work. Use a feature branch and a pull request for each meaningful feature.

Important: If an implementation decision is unclear, stop and ask the listed question instead of inventing a rule.

Architecture

Next.js 16 + React 19 + TypeScript + Tailwind 4 → /api/v1 → Supabase Auth + PostgreSQL + Storage + RLS.

2. Your exact ownership

You own	You support
Authentication and role routing	Shared database/schema
RLS and API guards	Nattu's frontend integration
HOD request inbox + decision page	Storage integration
HOD records + events	Student-side testing
Activity approvals	Shared integration testing
Review scheduler + attendance + notes	Reports data verification
Reports and exports	—

3. Before you start — confirm these

Questions to ask before coding

- ■ Is there exactly one HOD account, or will faculty/advisers also approve?
- ■ Should OD time use exact from/to times only, or also Full Day / Forenoon / Afternoon / Period 1–7?
- ■ Should weekly reviews remain strictly non-evaluative, with no scores?
- ■ What file types and maximum upload size should the department allow?
- ■ Confirm the current NAAC/NBA criteria before hard-coding any report labels or numbers.

These are explicitly identified as open decisions in the API contract/PRD. Do not silently decide them in code.

4. Phase 0 — Foundation

Do this with Nattu. The schema and API contract are shared and must not drift.

Prompt 0A — inspect the existing repository

```
You are helping me implement the SIET CSE Department Platform from the attached PRD v2.0 and
API Contract v2.0.
First inspect the repository. Do NOT change code yet.
Identify:
1. current Next.js structure,
2. existing auth/localStorage code,
3. DataContext/mock API calls,
4. existing /hod routes,
5. existing shared components,
6. places where hard-coded student identity or HOD identity exists,
7. current TypeScript types.
Return a concise migration checklist grouped into Auth, API, HOD UI, and Shared code.
```

```
Do not invent files that are not present.
```

Done when

- ■ You know where the current mock session is implemented.
- ■ You know where /hod routes and HOD components live.
- ■ You have identified all places that must stop using localStorage identity.

Prompt 0B — agree the API boundary

```
Using the API Contract v2.0 as the source of truth, list the endpoints the HOD portal will
consume in Phase 1.
For each endpoint give: method, path, role, request body, response shape, and validation rules.
Do not create new endpoints unless you explicitly mark them as a proposed change.
```

5. Phase 1 — Authentication + HOD Requests

Prompt 1A — Supabase auth and role routing

```
Implement Supabase Auth for the existing Next.js application.
Requirements:
- Replace mock login/localStorage session.
- Use the authenticated Supabase session.
- Read role from the users table; never accept role from the client.
- Implement GET /api/v1/me.
- Route STUDENT to /student and HOD to /hod.
- Wrong-role routes must redirect/deny access.
- Keep the existing UI design.
Before changing files, show the files you will modify and explain the auth flow.
```

Done when

- ■ A real HOD can sign in.
- ■ Role comes from the database, not a dropdown or request body.
- ■ /hod works for HOD and is denied for a student.
- ■ Refreshing the browser keeps the authenticated session.

Prompt 1B — RLS and server-side HOD guard

```
Implement database-enforced authorization for the HOD portal using Supabase RLS and server-side
role checks.
Requirements:
- HOD can read CSE departmental records allowed by the schema.
- Student cannot read another student's private records.
- Students cannot write APPROVED, REJECTED, or REVISION_REQUESTED states.
- Status history and audit logs are append-only.
- Client route protection is not the only security layer.
Show the SQL policies before applying them and explain each policy.
```

Done when

- ■ Policies work when queried directly, not only through the UI.
- ■ A student API request cannot perform an HOD decision.
- ■ Status history cannot be edited/deleted by users.

Prompt 1C — HOD OD inbox

```
Implement the HOD OD request inbox using the existing design system.
Use GET /api/v1/od-requests.
Support filters for status, academic year, section, category, and search where the existing
contract/schema supports them.
Do not redesign the whole dashboard.
```

```
Show compact rows and a clear empty state.
Then implement the request detail page using GET /api/v1/od-requests/{id}.
Use real API data, not mock data.
```

Prompt 1D — HOD decision endpoint

```
Implement POST /api/v1/od-requests/{id}/decision according to API Contract v2.0.
Allowed decisions:
- APPROVED: remarks optional
- REJECTED: rejection_reason required
- REVISION_REQUESTED: revision_notes required
The operation must atomically update the request, write od_status_history, create the relevant
notification, and record the audit event where required.
Do not allow raw client updates to the status column.
Return RFC 7807-style errors for validation failures.
```

Done when

- ■ Approve changes the status and history together.
- ■ Reject without a reason is blocked.
- ■ Revision without notes is blocked.
- ■ Student receives a notification after the decision.
- ■ A direct student request to the decision endpoint is rejected.

6. Phase 2 — Records, Events, Conflicts

Prompt 2A — Records directory

```
Implement the HOD Records page.
Use:
GET /api/v1/records/students
GET /api/v1/records/students/{id}/summary
GET /api/v1/records/export
Provide year/section/search filters, a student list, a slide-over summary, and CSV export.
Do not load all students into the browser if the API is paginated.
```

Prompt 2B — OD conflict detection

```
Implement OD conflict detection according to API Contract v2.0.
Check existing APPROVED ODs across dates, times, and included team members.
Show a clear warning to the HOD before approval.
Do not automatically reject; the HOD remains the decision-maker.
```

Prompt 2C — Events

```
Implement HOD event management using the existing event schema and UI.
An event should group its related OD requests and expose approved/pending/rejected counts.
Use database data only. Do not create a second event model in the frontend.
```

7. Phase 3 — Activity approvals

Prompt 3 — HOD activity approval

```
Implement the HOD side of Projects, Hackathons, and Internships.
Use GET /api/v1/activities and POST /api/v1/activities/{id}/decision.
Support the contract's lifecycle and decisions:
ACTIVE, REJECTED, REVISION_REQUESTED.
Show submitted documents inline where supported.
Keep the same decision principles as OD: human decision, recorded reason/notes, full history.
```

8. Phase 4 — Weekly Reviews

Prompt 4 — HOD review scheduler

```
Implement the HOD weekly review management.
Use:
POST /api/v1/reviews/schedule
GET /api/v1/reviews/sessions
POST /api/v1/reviews/sessions/{id}/generate-qr
POST /api/v1/reviews/sessions/{id}/finalize
Support the agreed recurring schedule type.
Show attendance and student progress.
Reviews are non-evaluative progress meetings unless the team explicitly resolves the open
scoring question first.
```

9. Phase 5 — Reports

Prompt 5 — accreditation reports

```
Implement the HOD accreditation report page using GET /api/v1/reports/accreditation.
Do not hard-code counts.
Read metrics from validated database records.
Add CSV and printable/PDF export only after the report data is verified.
Keep criteria labels configurable/verified rather than assuming the old hard-coded numbers are
current.
```

10. Integration checkpoints with Nattu

- ■ After Auth: give Nattu the final /api/v1/me response shape.
- ■ After OD API: give Nattu the exact POST, GET, resubmit, decision, and error shapes.
- ■ After Documents: give Nattu the upload and signed-URL behavior.
- ■ After Activities: give Nattu the activity response shape.
- ■ After Reviews: give Nattu session/progress/check-in response shapes.
- ■ Never change a shared response field silently; discuss it first.

11. Final Meena end-to-end test

Use a real HOD account and a real student account. The platform is not 'done' because the HOD screen looks correct.

Done when

- ■ Student submits an OD and it appears in the HOD inbox.
- ■ HOD approves/rejects/requests revision and the database records the transition.
- ■ A student cannot call the HOD decision endpoint successfully.
- ■ Records show real students across years/sections.
- ■ Conflict warnings use dates/times/team members.
- ■ Documents remain private and are accessed through signed URLs.
- ■ Review sessions, attendance, notes, and QR flows work end-to-end.
- ■ Reports calculate from database rows.

12. If you get stuck: use this prompt

Universal debugging prompt

```
I am implementing the SIET CSE Department Platform.
Here is the task: [TASK]
Here is the exact error/output: [ERROR]
Here is the relevant file/code: [PASTE]
```

```
Use the PRD v2.0 and API Contract v2.0 as the source of truth.
First explain the root cause in simple terms.
Then give the smallest safe fix.
Do not change the database contract, status machine, or permissions unless you explicitly
identify the change and ask me for confirmation.
After the fix, give me 3 tests to verify it.
```